CS 12 Cheatsheet

Diogn Lei R. Mortera

202508438 · Seat 81 · TCD/FRU2

Commands

```
#pyright : strict
from __future__ import annotations
from dataclasses import dataclass
from enum import Enum, auto
if __name__ == '__main__': main()
```

git `push, pull, commit -m '<msg>', add`

pytest `coverage run --batch -m pytest, coverage html`

pyxel `pyxel run <file>, pyxel package . <fname>.py, pyxel app2html <fname>.pyxapp`

Introductory Concepts

type hinting (`:,->`), inference (`Pyright`), narrowing, aliases sub/supertypes, long-lived programs, exhaustiveness checking

model-view-controller/presenter model - game logic, view - display logic (terminal, `pyxel`), controller - unify model and view, game loop

pre/postconditions conditions before/after function is called

invariant condition that is true throughout entire lifetime of object (and by LSP, its subclasses)

inversion of control flow of control given to code by framework (e.g. food service that delivers healthy food to house every week)

search space reduction split code into units for easier unit testing

Testing and Debugging

input/output equivalence partitions, edge cases, boundary values, search space reduction, unit/stress/fuzz testing, `pytest`

non-throwaway code bug-free, easy to understand, easy to change; desired outcome of testing

test coverage % parts of code executed by **test suite**, 100% coverage $\neq$ no bugs

line/branch coverage lines / t/f conditionals in assembly code

Modularization

module, modularization smaller, reusable components

separation of concerns nonoverlapping aspects of code (related (but not) S in SOLID)

dependency baked-in (hard coded), injected (e.g. Strategy patterns), decoupled code easier to change

SOLID

single responsibility separation of concerns, gathering of things that change for same reason

open closed open for extension, closed for modification

Liskov substitution behavioral subtyping, history rule, pre/postconditions, Barbara Liskov

interface segregation can separate interfaces, implicit (`Protocol`) and explicit (`ABC` / abstract base class)

dependency inversion depend on interfaces and not concrete classes

Subtyping

subtype polymorphism, hierarchical substitutability of types

subtyping hierarchy ABC, exception, numeric tower

`from collections.abc from collections`

covariance subtype of complex follows simple

contravariance subtype of complex is the reverse of simple

bivariance either direction applies

invariance no trend

Basic Subtyping

```
<: - preorder, reflexive, transitive
A <: A
A <: A | B, B <: A | B
A <: B and B <: C -> A <: C #transitive
A <: object
```

Container Subtyping

```
Drink <: Liquid
ROL[Drink] <: ROL[Liquid] #read-only coV
WOL[Liquid] <: WOL[Drink] #write-only contraV
```

Functional Subtyping

```
Number = float | int #let

perform_operation(f : Callable[[Number],
Number]):
  return f(a,b)

Let Num = int | float
int <: Num

f(x : Num) <: f(x : int) #parameter
contravariance
f -> int <: f -> Num #return covariance
```

LSP Rules

contravariant method parameters and **covariant return types**

exceptions thrown subtypes of original (e.g. if class has no exception, then its subclass must have none too)

no strengthening preconditions don't require stricter than originally needed (e.g. if superclass demands ≥ 0, subclass should not demand > 0)

no weakening postconditions (e.g. if superclass provides nonempty deck, subclass must not provide empty deck)

no violating invariant must not violate behavior based on specified invariants (in docstrings), including interactions and calling (or not calling) exceptions, staying true to roots

no violating history rule any state transition of child has to be possible using parent's methods (e.g. if you can create some method that uses other methods of a parent to perform that task)

Object-Oriented Programming

(networks of message-sending objects)

abstraction ABCs, explicit interface, implementation hidden

encapsulation one concern - one responsibility, each object featuring attributes and methods that change properties of that class

polymorphism subtype polymorphism, subclasses

inheritance from ABCs or instantiated classes to create derived classes

composition objects referencing other objects; dependency injection

static method, static factory method

forwarding passing message to another object

more objects, more cohesive yet complex

maximize cohesion, minimize coupling

public interface `@property`, non _-methods

Note: _-prefixed vars are *protected*, NOT *private* (e.g. can't be accessed except by children)

OODP

object-oriented design patterns

creational abstract factory, builder, factory method, prototype, singleton

structural *adapter*, bridge, composite, decorator, facade, flyweight, proxy

behavioral chain of responsibility, command, iterator, interpreter, mediator, memento, *observer*, *state*, *strategy*, *template method*, visitor

Structural

adapter incompatible (contained in adapter) + compatible interface (implemented in adapter) + adapter class (conversion logic)

Behavioral

observer allows observable to notify observers without knowing who observers are

- Observer = Subscriber = Listener
- Observable = Publisher

state sole purpose is to change the state (good with `itertools.cycle`), Enum-based entries has `next(self)` function

strategy introduces behavior via composition, has (a) strategy interface (e.g. `GuessingGameStrategy`), (b) concrete class (e.g. `BinarySearchStrategy`), and (c) context class (e.g. `Player(s : Strategy)`)

template method delegates implementation of *some* steps to subclassses, usually ABCs

Game Programming

graphical, nonblocking I/O, game loop, inversion of control, FPS, normalize speed by dividing by FPS, use MVC

Essentials

```
pyxel.init(size_x, size_y, fps=fps)
def update(): ... # game state
def draw(): ...# screen
pyxel.run(update, draw)

.mouse(True)
```

Drawing

```
.text(x,y,text,color)
.circ(x,y,r,color)
.rect/.rectb(x,y,w,h,color)
.cls(color)
.mouse_x .mouse_y .mouse_wheel
KEY_{CHAR}{NUMBER}{UP/DOWN/LEFT/RIGHT}
KEY_RETURN
```

Running

```
pyxel run <file> | pyxel package . <fname>.py |
pyxel app2html <fname>.pyxapp
```

HTML / DOM

HyperText Markup Language (HTML) defines how web pages are structured

document object model (DOM) a tree representing the structure of an HTML page in tree form

open-close / self-closing tags, inline (take necessary space) and block (takes entire page width) elements

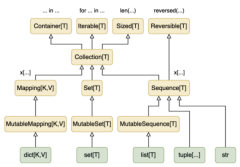
scripting language to manipulate the DOM tree (e.g. `JS`, `TypeScript`, `PyScript`)

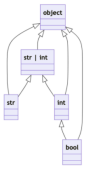
```
<html><head><title><body>
<p><div><h1/2/../6> <!-- block elements -->
<span><input> <!-- inline elements -->
document.createElement()
.getElementById('#id') .querySelector('.class')
.textContent .innerHTML
```

References

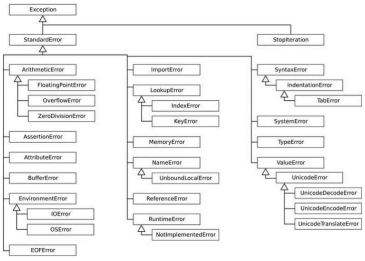
lower <: higher (lower can substitute for higher)

Types

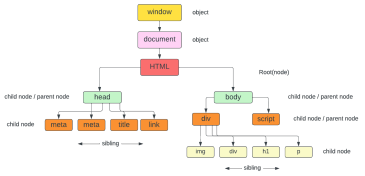




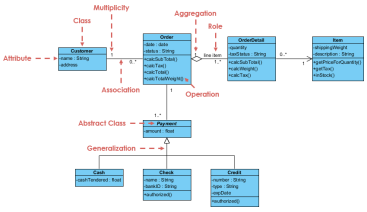
Exceptions



DOM



UML




```
#pyright : strict
import pytest
import pyxel
from random import Random
from collections.abc import Sequence
from abc import ABC, classmethod
from typing import Protocol
from enum import Enum, auto
from dataclasses import dataclass
from pyscript import document, web
```

```
from typing import TypeVar
def repeater(value : T) -> list[T]: #type of
T depends on param input, but list[T] is
inferred to be a list of that type
  return [value for _ in range(5)]
```

```
model.py | view.py | controller.py
controller =
CalcController(CalcModel(),CalcView())
# put most print() / draw() in V
# put most logic in M
# call and pass methods between M & V in C
```

Basic Type Hints

```
bool | int | str | Literal['mkns','mkbn'] |
list[int] | dict[int, str] | Callable[[int],
int]
tuple(1,2,3) : tuple[int,int,int]
tuple(varying size) : tuple[int, ...]
Grid = list[list[str]] # type alias


# f(g(x)) type-checks if return(g) <:
param_type(f)


#ILLEGAL
list[int, ...] #... only for immutable
containers (e.g. tuple[int, ...])
```

Subtyping "Why"s

```
ROL[Drink] <: ROL[Liquid] # drink can do
anything a liquid can (history rule) |
COVARIANCE
ROL[Liquid] </: ROL[Drink] # not all liquids
can be drank


WOL[Liquid] <: WOL[Drink] # only cares if
things inside are its subtypes |
CONTRAVARIANCE
WOL[Drink] </: WOL[Liquid] # wanna put gas in
your drink list?
```

Parameter Contravariance

```
f (x : Number) <: f (x : int) #let addition
be an f(x : Number). type safety maintained
because int's can be added as it is a subtype
of Number


f (x : int) </: f (x : Number) #let modulo be
an f(x : int). then some numbers cannot be
modulo-ed (like floats)
```

Return Covariance

```
f -> int <: f -> Number # an int can still
perform everything expected of a Number (e.g.
+, -, *)
f -> Number </: f -> int
# will `return list[f]` still work if f ->
Number, which can be a float?
```

Usual Liskov Suspects

making types of subclasses parameters more specific · subclasses override superclass · errors not carried over in subclasses · different / unpassed parameters from sub/superclass · invariant being overridden · inadvertent behaviors despite invariant statement · narrowing parameter types in overriden methods · **name mangling** issues · forgetting `super().__init__()` · · ·

OODP Samples

Strategy

```
class Human:
  def __init__(love_language :
LoveLanguage): ...
class LoveLanguage(Protocol): ...
# classes below realizes interface above
class TimeLoveLanguage: ...
class TouchLoveLanguage: ...

time_lover = Human(TimeLoveLanguage)
touch_lover = Human(TouchLoveLanguage)
```

Simple Factory

```
LoveLanguageKind = Enum('LoveLanguageKind',
'TIME TOUCH')

class LoveLanguageFactory:
  @classmethod
  def make(cls, ll_kind : LoveLanguageKind):
    match ll_kind:
      case LoveLanguageKind.TIME:
        love_language = TimeLoveLanguage()
      case LoveLanguageKind.TOUCH:
        love_language = TouchLoveLanguage()


    return Human(love_language)
```

Template Method

```
class Biscuit(ABC):
  def __init__(self):
  def take(self): print('Take it')

  @abstractmethod
  def break(self): ...

  @abstractmethod
  def taste(self): ...

class Cookie(Biscuit):
  def break(self): print("Don't break it")
  def taste(self): print("I wanna see you
taste it")
```

State

```
class Feeling():
  def __init__(self):
    self._state = HappyState(self)

  def change_state(self, state :
FeelingState):
      self._state = state
  def next(self):
      self._state = self._state.next()
      return self._state

class FeelingState(ABC):
  def __init__(self, f : feeling):
    self._feeling = f

  def set_state(self, state : FeelingState):
    self._feeling.change_state(state)
  @abstractmethod
  def next(self): pass

class HappyState(FeelingState):
  def next(self):
      print('S')
      new_state = SadState(self._feeling)
      self.set_state(new_state)
      return new_state
class SadState(FeelingState):
  def next(self):
    print('H')
      new_state = SadState(self._feeling)
      self.set_state(new_state)
      return new_state
```

Observer

```
@dataclass
class Post:
  post : str = 'Empty post'
  date : Date =
field(default_factory=Date.now())


# in this example, the SocialMediaAccount is
both the observable and the observer.
class SocialMediaAccount:
  def __init__(self, posts : Sequence[Post]):
    self._posts : list[Post] = posts
    self._followers : set[SocialMediaAccount]
= {}
    self._feed : list[Post] = []

  # some changes on an observable, like
add_post, notify / update the observers
  def add_post(self, post : Post):
    self._posts.append(post)
    self.update_followers(post)


  # the observer "subscribes" to the
observable.
  def follow(self, account :
SocialMediaAccount):
    self._follow(account)
    account.add_follower(self)


  def add_follower(self, account :
SocialMediaAccount):
    self.add(account)


  # the observable is updating all of its
"observers"
  def update_followers(self, post : Post):
    for follower in self._followers:
      follower.update_feed(post)


  def update_feed(self, post : Post):
    self._feed.append(post)
```

Adapter

```
@dataclass
class Vector:
    magnitude: float
    direction: float  # radians


@dataclass
class VectorComponent:
    x: float
    y: float


    def get_components(self) ->
VectorComponent: ...


class VectorToVectorComponentAdapter(
  VectorComponent):
    def __init__(self, vector: Vector):
        self._vector = vector


    def get_components(self) ->
VectorComponent:
        x = self._vector.magnitude *
math.cos(self._vector.direction)
        y = self._vector.magnitude *
math.sin(self._vector.direction)
        return VectorComponent(x, y)


v = Vector(50, math.pi / 2)
adapter = VectorToVectorComponentAdapter(v)


print( adapter.get_components() ) # outputs
around (0, 50)
```

Game Sample

```
import pyxel
from dataclasses import dataclass
FPS = 60
SPEED = 10
DT = SPEED / FPS

class World: WIDTH = 200; HEIGHT = 200

@dataclass
class Ball:
    x : float ; y : float ; r : float; v_x :
float; v_y : float; g : int = 10


    def update(self):
        self.x += self.v_x * DT
        self.v_y += self.g * DT
        self.y += self.v_y * DT


    def collide_x(self):
        if not (self.r <= self.x <=
World.WIDTH - self.r):
            self.v_x = -self.v_x


    def collide_y(self):
        if not (self.r <= self.y <=
World.HEIGHT - self.r):
            self.v_y = -self.v_y


    def jump(self): self.v_y = -20

ball = Ball(World.WIDTH // 2, World.HEIGHT //
2, 5, 10, 0)

def update():
    ball.update(); ball.collide_x();
ball.collide_y()
    if pyxel.btnp(pyxel.MOUSE_BUTTON_LEFT):
        ball.jump()

def draw():
    pyxel.cls(0)
    pyxel.circ(ball.x, ball.y, ball.r, 5)

pyxel.init(World.WIDTH,World.HEIGHT)
pyxel.mouse(True)
pyxel.run(update, draw)
```

HTML / DOM Sample

```
from pyscript import document, web
#warning: better to use MVC, decreases
'global' spam
root = document.getElementById("app")

count = 0

def handle_incr(ev):
    global count
    print("+ was clicked")
    count += 1
    ctr.innerText = str(count)

def handle_decr(ev):
    global count
    print("- was clicked")
    count -= 1
    ctr.innerText = str(count)

incr_btn = document.createElement("button")
incr_btn.innerText = "+"
incr_btn.addEventListener("click",
web.create_proxy(handle_incr))

ctr = document.createElement("p")
ctr.innerText = "0"

decr_btn = document.createElement("button")
decr_btn.innerText = "-"
decr_btn.addEventListener("click",
web.create_proxy(handle_decr))

root.appendChild(incr_btn)
root.appendChild(ctr)
root.appendChild(decr_btn)
```

Bonus Questions

Labs

Lab 0-1: CS11 Recall, Type Hinting · Lab 2: Battleships · Lab 3: Whac-A-Mole: Terminal Version · Lab 4: Connect-Tac-Toe · Lab 5: Berry Interesting Pokémon Battle

Instructors

Jerome Cary Beltran, Daryll Carlsten Ko, Miguel Martinez, Jozelle Addawe, Ren Tristan Dela Cruz, Mario Carreon, ~~Juan Felipe Caronel~~

Student Assistants

Vaughn Aquino · Eliana Mari Lim · Charlize Sim · Gabby Sacramento · Aeron Dann Peñaflorida · Jarelle Gail Ricaforte · Lucas Agcaoili · Marius Ulyzses Barcenas · Tristan Jovan Noval

Slide Titles

(0-indexed) Hi! · Static Type Checking (in Python) · Debugging and Testing · Separation of Concerns · Subtyping · Object-Oriented Programming and Design · Single Inheritance · Type Variance and Function Subtyping · Behavioral Subtyping · Object Oriented Design Patterns · Game Programming Basics · Game Programming II (MVC) · HTML and the DOM (PyScript)

Credits

- Diogn Mortera (majority of cheatsheet + code for OODP and LSP)
- Jared Lising, Justin Camacho (help on LSP checklist)
- Jerome Cary Beltran (`pyscript` code for Lec 13)
- Sorsogon Krazy Cat's cheatsheet for double-checking
- CS 12 25.1 and 25.2 Slides
- certain people for Motivational Quotes ;)

Motivational Quotes

"miss ko na siya" *me*
"Take it, don't break it, I wanna see you taste it" *Cookie, NewJeans*
"It will pass" *my speech30 seatmate*
"It can't be that hard" *djmc*
"Ah, 'di na malabo" *LASIK, HEY JUNE!*
"Truth emerges more readily from error than from confusion" *Thomas Kuhn, The Structure of Scientific Revolutions*
"am i gonna cook or im going to be cooked" *yn*
"If only we knew the suffering that would befall us next..." *ULTRAKILL*
"What makes a test good or bad? The most basic and obvious answer to that question is that good tests measure what you want to measure, and bad tests do not." *Ben Clay, Is This a Trick Question?*
"Reviewing is doubting your potential." *ngrse*
"It hurts to be something, it's worse to be nothing with you" *Promise, Laufey*
"I give 98 percent of my mental energy to CS12. Others give only 2 percent." *Bobbeyuowan Fisher*
"One who bows to the sweeping winds will be swept away themselves: only by opposing the breeze will one reach the treasure it guards so tightly" *justin*
"Momentum without sustainability is death" *World Economic Quorum*
"'Cause there is something and there is nothing, there is nothing in between" *Ricky Montgomery, Line Without a Hook*